Lecture 6
Cube Computation

CSC1104: Data Warehousing and Data Mining
2026/2027 Semester 1

Prof. Mark Roantree
COMSCI4 @ Dublin City University



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Agenda



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

1. **Efficient Computation of Data Cubes**
   - Understanding Dimensionality
   - Partial Materialisation
2. **Methods for Data Cube Computation**
   - Full Cube Materialisation
   - Iceberg Cube Materialisation
   - Closed Cube Materialisation
3. **Bottom Up Construction**

Overview



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

- At the core of multidimensional data analysis is the efficient computation of aggregations across many sets of dimensions.
- In SQL terms, these aggregations are referred to as group-by's.
- Each group-by can be represented by a cuboid, where the set of group-by's forms a lattice of cuboids defining a data cube.
- It is important to understand issues relating to the efficient computation of data cubes.

The Curse of Dimensionality



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

- One approach to cube computation extends SQL so as to include a **compute cube** operator.
- The compute cube operator computes aggregates over all subsets of the dimensions specified in the operation.
- This can require excessive storage space, especially for large numbers of dimensions.
- We start with an intuitive look at what is involved in the efficient computation of data cubes.

Example

- A data cube is a **lattice of cuboids**.
- Suppose you need to create a data cube for *AllElectronics* sales that contains the following: *city*, *item*, *year*, and *sales* in dollars.
- This enables analyses such as:
  - Compute the sum of sales, grouping by city and item.
  - Compute the sum of sales, grouping by city.
  - Compute the sum of sales, grouping by item.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Calculating the Cuboids

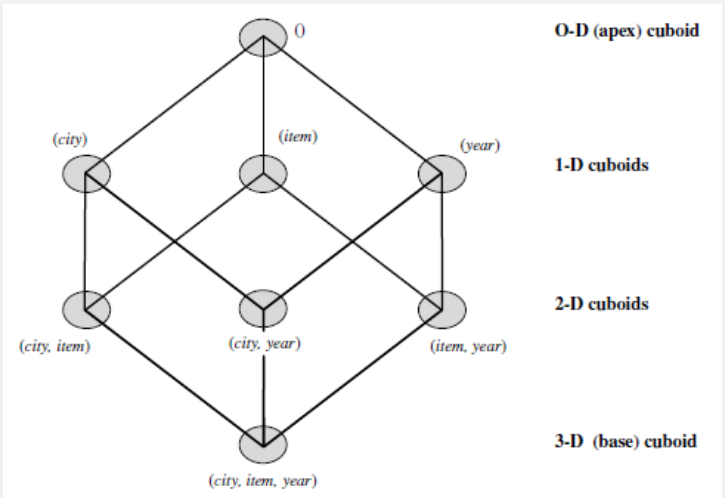
- What is the total number of cuboids, or group-by's, that can be computed for this data cube?
- Taking the three attributes, *city*, *item*, and *year*, as the dimensions for the data cube, and *sales* as the **measure**, the total number of cuboids, or group by's, that can be computed for this data cube is: $2^3 = 8$.
- The possible group-by's are the following: {(city,item,year), (city,item), (city,year), (item,year), (city), (item), (year), ()}, where () means that the group-by is empty (i.e. the dimensions are not grouped).



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Forming the Lattice

- These group-by's form a lattice of cuboids for the data cube, as shown in Figure 1.
- The base cuboid contains all three dimensions (*city*, *item*, and *year*) returning the total *sales* for any combination of the three dimensions.





Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Lattice Components

- The **apex cuboid**, or *0-D* cuboid, refers to the case where the group-by is empty. It contains the total sum of all sales.
- The **base cuboid** is the least generalised (most specific) of the cuboids.
- The apex cuboid is the most generalized (least specific) of the cuboids, and is often denoted as **all**.
- If we start at the apex cuboid and explore downward in the lattice, this is equivalent to drilling down within the data cube.
- If we start at the base cuboid and explore upward, this is the process of rolling up.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Dimensionality and Group By

- The SQL query containing no **group-by**, such as *compute the sum of total sales*, is a *zero*-dimensional operation.
- The SQL query containing one **group-by**, such as *compute the sum of sales, group by city*, is a *one*-dimensional operation.
- A cube operator on n dimensions is equivalent to a collection of **group by** statements, one for each subset of the n dimensions.
- Therefore, the **cube** operator is the n-dimensional generalisation of the **group by** operator.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Define Cube

The data cube for this example could be defined as:

$$define\ cube\ sales_cube[city, item, year] : sum(sales) \quad (1)$$

For a cube with n dimensions, there are a total of 2^n cuboids, including the base cuboid. A statement such as

$$compute\ cube\ sales_cube \quad (2)$$

would explicitly instruct the system to compute the sales aggregate **cuboids** for all of the eight subsets of the set *{city,item,year}*, including the empty subset.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Precomputing Cubes

- OLAP analyses often access the same cuboids for different queries.
- Therefore, it may seem like a good idea to compute all or at least some of the cuboids in a data cube in advance.
- Precomputation leads to fast response times and avoids some redundant computation.
- Most, if not all, OLAP products resort to some degree of precomputation of multidimensional aggregates.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Curse of Dimensionality

- A major challenge related to this precomputation is the required storage space: if all of the cuboids in a data cube are precomputed, especially when the cube has many dimensions.
- The storage requirements are even more excessive when many of the dimensions have associated concept hierarchies, each with multiple levels.
- This problem is referred to as the **curse of dimensionality**.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Issues of Scale

- The size of each cuboid also depends on the **cardinality** (number of *distinct values*) of each dimension.
- For example, if the *AllElectronics* branch in each city sold every item, there would be $|\ city\ | \times |\ item\ |$ tuples in the *city-item* group-by alone.
- In the example, the sales amount is stored for every product, for every city.
- As the number of dimensions, number of conceptual hierarchies, or cardinality increases, the storage space required for many of the group-by's will grossly exceed the (fixed) size of the input relation.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Working Example

A	B	C
Month	*Part*	*City*
Jan	P1	Toronto
Feb	P2	Vancouver
Mar	P3	Montreal
Apr	P4	
AB Cuboids (16)		
{Jan,P1,*}		
{Jan,P2,*}		
{Jan,P3,*}		
{Jan,P4,*}		
{Feb,P1,*}		
{Feb,P2,*}		
{Feb,P3,*}		
{Feb,P4,*}		
.....		
{Apr,P4,*}		
AC Cuboids (12)		
BC Cuboids (12)		
ABC Cuboids (48)		

Figure 2: Dimensionality Problem: Every *instance* of AB cuboid



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Understanding Scale

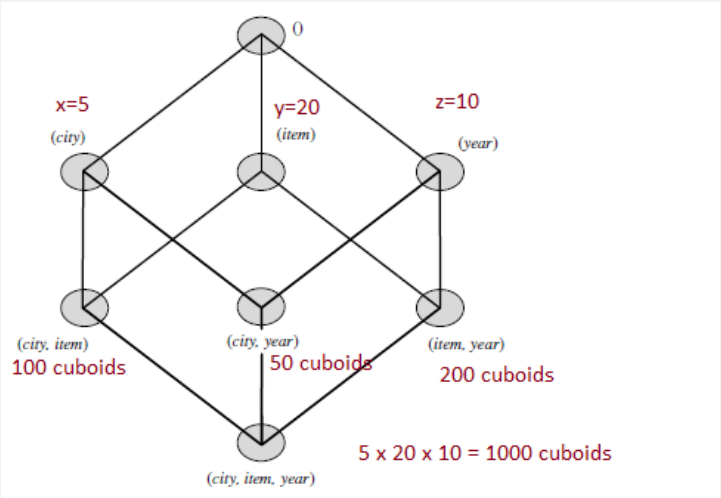


Figure 3: Many (potential) cuboids but not actual cuboids!



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Choices for Data Cube Materialisation

1. **No materialisation**. Do not precompute any of the *nonbase* cuboids. This leads to computing expensive multidimensional aggregates on the fly, which can be extremely slow.
2. **Full materialisation**. Precompute all of the cuboids. The resulting lattice of computed cuboids is referred to as the full cube and requires huge amounts of memory space.
3. **Partial materialisation**. Selectively compute a proper subset of the whole set of possible cuboids. Alternatively, we may compute a subset of the cube, which contains only those cells that satisfy some user-specified criterion. We will use the term **subcube** to refer to this case where only some of the cells may be precomputed for various cuboids.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Forming the Lattice

- A cell in the base cuboid is a **base cell**.
- A cell from a non-base cuboid is an **aggregate cell**.

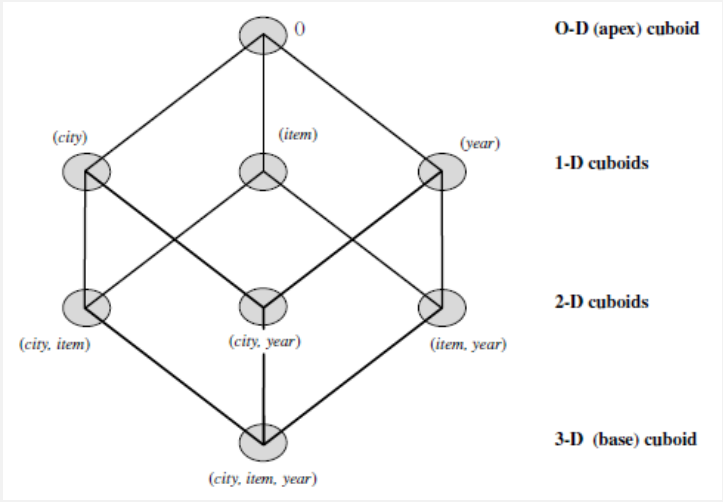


Figure 4: Level 3 are base cells; levels 0,1,2 are aggregate cells



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Cube Structure

- An aggregate cell aggregates over one or more dimensions, where each aggregated dimension is indicated by a "*" in cell notation.
- Assume we have an n-dimensional data cube and let $a=(a_1,a_2,\ldots,a_n$ measures) be a cell from one of the cuboids making up the data cube.
- We say that a is an m-**dimensional cell** (from an m-dimensional cuboid) if exactly $m(m \leq n)$ values among $\{a_1,a_2,\ldots,a_n\}$ are not "*".
- If $m = n$, then a is a **base** cell; otherwise, it is an **aggregate** cell (where $m < n$).



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

1-D Sample Cuboids

A	B	C	count()			ALL
*	*	*	100		apex	
Jan	*	*	20			
Feb	*	*	25			
Mar	*	*	30			
Apr	*	*	?		Only 4 months of data so (Apr,*,*) must be?	
*	Toronto	*	35			
*	Vancouver	*	?		Only 3 cities so (*,Vancouver,*) must be?	
*	Montreal	*	40			
*	*	P1	25			
*	*	P2	25			
*	*	P3	25			
*	*	P4	25			

Figure 5: Different forms of 1-D cells (plus 0-D at thetop).



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

2-D and 3-D Sample Cuboids

Jan	Toronto	*	12		
Jan	Vancouver	*	8		
Feb	Toronto	*	10		
Feb	Vancouver	*	2		
Feb	Montreal	*	13		
Mar	Toronto	*	10		
Mar	Montreal	*	20		
Apr	Toronto	*	3		
Apr	Vancouver	*	15		
Apr	Montreal	*	7	100	What does that mean for (Jan, Montreal,*)?
					There will also be the AC and BC combinations!
Jan	Toronto	P1			What is the max this can be?
Apr	Vancouver	P1	3		
Apr	Vancouver	P2	5		
Apr	Vancouver	P3	4		
Apr	Vancouver	P4	?		How many?

Figure 6: Month x City 2-D cells and Month x City x Product 3-D (base) cells.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Example: Base and Aggregate Cells

- Consider a data cube with the dimensions *month*, *city*, and *customer group*, with the measure *price*.
  - (Jan,*,*,2800) and (*,Toronto,*,1200) are 1-D cells;
  - (Jan,*,Business,150) is a 2-D cell;
  - and (Jan,Toronto,Business,45) is a 3-D cell.
- Here, all base cells are 3-D, whereas 1-D and 2-D cells are aggregate cells.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Ancestor and Descendant cells

An ancestor-descendant relationship may exist between cells:

- $a = (Jan, *, *, 2800)$ (1-D cell)
- $b = (Jan, *, P1, 150)$ (2-D cell)
- $c = (Jan, Toronto, P1, 45)$ (3-D cell)
- The 1-D and 2-D cells are ancestors of the 3-D cell.
- c is a descendant of both a and b;
- b is a parent of c,
- and c is a child of b.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Not Ancestors or Descendants!

Referring to our previous example,

- 1-D cell $a = (Jan, *, *, 2800)$;
  2-D cell $b = (Feb, *, P2, 150)$;
  3-D cell $c = (Jan, Toronto, P2, 45)$;
- c is a descendant of a but not b;
- b is not a child of a,
- c is not a child of b.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Materialisation and Sparse Cubes

- In many cases, a substantial amount of the cube's space is consumed by a large number of cells with very low measure values.
- This is because the cube cells are often quite sparsely distributed within a multi-dimensional space.
- For example, a customer may buy few items at a time, generating a small number of non-empty cells and leaving most cells empty.
- If a cube contains many sparse cuboids, the cube is **sparse**.
- In such situations, it is useful to materialize only those cells in a cuboid (group-by) whose measure value is above some minimum threshold.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Iceberg Cubes

- In a data cube for *sales*, we may wish to materialize only those cells for which *count* ≥ 3 (where at least 3 tuples exist for the cell's given combination of dimensions),
  or only those cells representing *sales* $\geq$ €100.
- This not only saves processing time and disk space, but also leads to a more focused analysis.
- The cells that cannot pass the threshold are likely to be too trivial to warrant further analysis.
- Such partially materialized cubes are known as **iceberg cubes**.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Understanding Sparsity

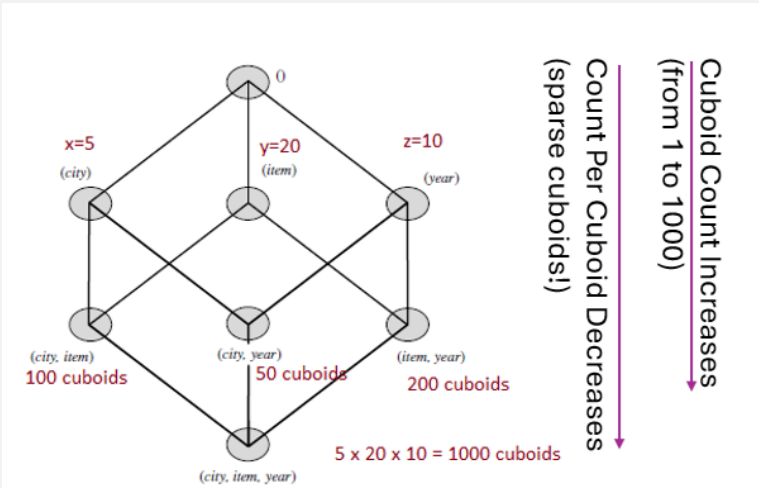


Figure 7: Many empty(ish) cuboids!



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Sample Iceberg using min_sup

- The minimum threshold is called the **minimum support threshold**, or *minimum support (min_sup)*.
- By materializing only a fraction of the cells in a data cube, the result is seen as the *tip of the iceberg*, where the **iceberg** contains *all cells*.
- An iceberg cube can be specified with the SQL query shown in the following example.

```
1 compute cube sales_iceberg as
2 select month, city, product, count(*)
3 from salesInfo
4 cube by month, city, product
5 having count(*) >= min_sup
```

Sample Iceberg Cube



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Compute Cube

- The **compute cube** statement specifies the precomputation of the iceberg cube *sales_iceberg*, with the dimensions *month*, *city*, and *product*, and the aggregate measure *count()*.
- The input tuples are in the *salesInfo* relation.
- The **cube by** clause specifies that aggregates (group-by's) are to be formed for each of the possible subsets of the given dimensions.
- When computing the full cube, each group-by corresponds to a **cuboid** in the data cube lattice.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Iceberg Condition

- The constraint specified in the **having** clause is known as the **iceberg condition**.
- Here, the iceberg measure is *count*.
- The iceberg cube computed here can also be used to answer group-by queries on *any combination* of the specified dimensions of the form **having count(*)** $\geq$ v, where v $\geq$ min_sup.
- Instead of *count*, the iceberg condition could specify more complex measures, such as *average*.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Full Computation

- If we omit the **having** clause, we compute the full cube.
- We will call this full cube *sales_cube*.
- The iceberg cube *sales_iceberg*, excludes all the cells of *sales_cube* whose count is less than *min_sup*.
- Note: if we set the minimum support to 1 in *sales_iceberg*, the resulting cube would be the full cube, *sales_cube*.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Many Cells: little usable data

- Even reducing the computation of trivial aggregate cells, we could still end up with a large number of uninteresting cells to compute.
- For example, suppose that there are **2 base cells** for a database of 100 dimensions, denoted as {(a1,a2,a3,...,a100):10, (a1,a2,b3,...,b100):10}, meaning each has a cell count of 10.
- If the minimum support is set to 10, there are too many cells to compute and store, with most unusable (*support* $<$ 10)
- For example, there are 2^{101}-6 distinct aggregate cells, like {(a_1,a_2,a_3,a_4,...,$_{99}$,*):10,...,
(a_1,a_2,*,a_4,...,$_{99}$,a_{100}):10,...,
(a_1,a_2,a_3,*,...,*,*):10},
but most of them will not meet *min_sup*.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Little new data: using the example

- We had: 4 months x 3 cities x 4 products = 4x3x4 = 48 possible base cells
- If count(apex) = 100, then on average, we have a count of 2 per cell, which means none are usable.
- The previous example assumes we have only 2 base cells where min_sup $\geq$ 10
- Assume these are:
{January, P1, Toronto} and {January, P1, Montreal}



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Dropping Uninteresting Data

Jan	P1	Toronto	10	
Jan	P1	Montreal	10	
Jan	P1	*	20	
Jan	P1	Vancouver		Any new information? Do we materialise?
If				
Jan	P1	*	29	Any new information?

Figure 8: Use the Aggregate Cells to determine *interesting* base cells. In this case answers are: No and No!



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Interesting Cells

- If we ignore all of the aggregate cells that can be obtained by replacing some constants by *'s while keeping the same measure value, there are only three distinct cells left:
  - {(a1,a2,a3,. . . ,a100):10,
  - (a1,a2,b3,. . . ,b100):10,
  - (a1,a2,*,. . . ,*):20} (the rollup).
- That is, out of 2^{101}-6 distinct aggregate cells, are useful datasets for analysis.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Or using our example . . .

- {(P1, Toronto, January):10,
- (P1, Toronto, March):10,
- (P1, Toronto, *):20}.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Closed Cube

- To systematically compress a data cube, we need to introduce the concept of **closed coverage**.
- Compress? Delete selected cuboids.
- A cell *c*, is a *closed cell* if there exists no cell *d*, such that *d* is a specialisation (descendant) of cell *c*, and *d* has the same measure value as *c*.
- **{P1, Toronto}=25** and {P1, Toronto, Jan}=15; {P1, Toronto, Feb}=5; {P1, Toronto, Mar}=5; {P1, Toronto, Apr}=0.
- *d* is a specialisation of *c* if *d* is obtained by replacing a * in *c* with a non-* value, and *d* has the same measure value as *c*.
- A **closed cube** is a data cube consisting of only closed cells.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Closed Cube Example

- For example, the three cells derived earlier are the **three closed cells** of the data cube for the data set: {(a_1,a_2,a_3,...,a_{100}):10, (a_1,a_2,b_3,...,b_{100}):10}.
- They form the closed cube lattice shown in Figure 9.
- Only materialise (and store) appropriate cuboids!

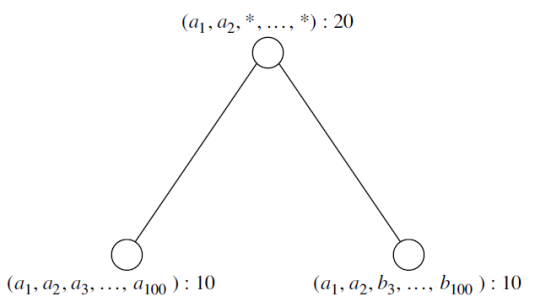


Figure 9: If either cell at the bottom has a count of 20, the ancestor is **not closed** and all other cells must be 0.

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Every other dataset can be derived!

- Other **non-closed cells** can be derived from their corresponding *closed* cells in this lattice.
- For example, "(a1,*,*,...,*):20" can be derived from "(a1,a2,*,...,*):20"' because the former is a generalised non-closed cell of the latter.
- Similarly, we have "(a1,a2,b3,*,...,*):10".
- If we know all missing cells have little new information (eg. = 0), then we can derive the Rollups.

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Closure and Eliminating Cuboids

min_sup=3			
Mar	P2	*	15
Mar	P1	*	10
Feb	P1	*	10
Jan	P1	*	11
Jan	P2	*	10
Jan	P1	Toronto	5
Jan	P2	Toronto	5
Feb	P1	Toronto	10
Mar	P1	Toronto	10
Mar	P2	Toronto	5

Figure 10: Which Rollups are dropped?

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Closure and Eliminating Cuboids

min_sup=3			
Mar	P2	*	15
Mar	P1	*	10
Feb	P1	*	10
Jan	P1	*	11
Jan	P2	*	10
Jan	P1	Toronto	5
Jan	P2	Toronto	5
Feb	P1	Toronto	10
Mar	P1	Toronto	10
Mar	P2	Toronto	5

Figure 11: 2 Rollups are dropped

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Summary

- Optimisation 1: Use Iceberg function to dismiss all cells that fail to meet threshold
- Optimisation 2 (Closure): If a Rollup fails to increase the Count, eliminate the Rollup cuboid.
- This has the effect of storing only the most specialised cuboids.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

BUC: Computing Iceberg Cubes from the Apex Cuboid Downward

- BUC is an algorithm for the computation of sparse and iceberg cubes.
- Algorithm constructs the cube *from* the **apex** cuboid toward the **base** cuboid.
- This allows BUC to share data partitioning costs.
- This order of processing also allows BUC to *prune* during construction, using the **Apriori** property.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Cuboid Lattice

- Lattice of cuboids: making up a 3-D data cube with dimensions *A*, *B*, and *C*.
- The apex (0-D) cuboid, representing the concept **all** (*,*,*), is at the **bottom** of the BUC lattice.

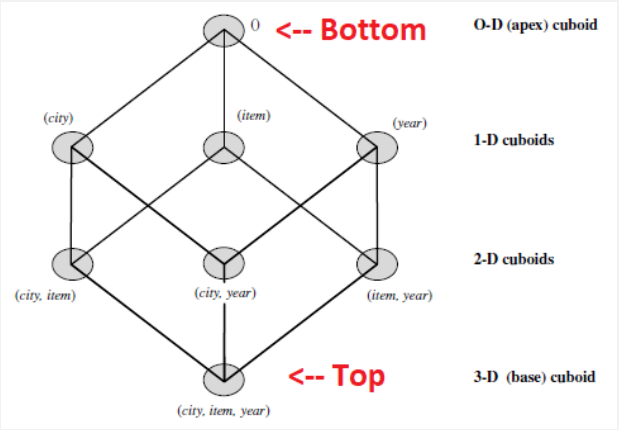


Figure 12: BUC flips this lattice upside down



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

BUC: Bottom-Up Construction

- The creators of BUC view a lattice of cuboids in the reverse order, with the apex cuboid at the bottom and the base cuboid at the top.
- In that view, BUC does bottom-up construction.
- We see drill-down as drilling from the apex cuboid down toward the base cuboid and thus, the exploration process of BUC is regarded as top-down.
- BUC's exploration for the computation of a 3-D data cube is shown in Figure 13.
- The exploration (order in which the lattice is traversed) starts with the highest count eg. Count(A) = 100; Count(B) = 85; Count(C) = 50.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

BUC Computation

Computation starts from the Apex Cuboid . . . moving from most aggregated towards base cuboids.

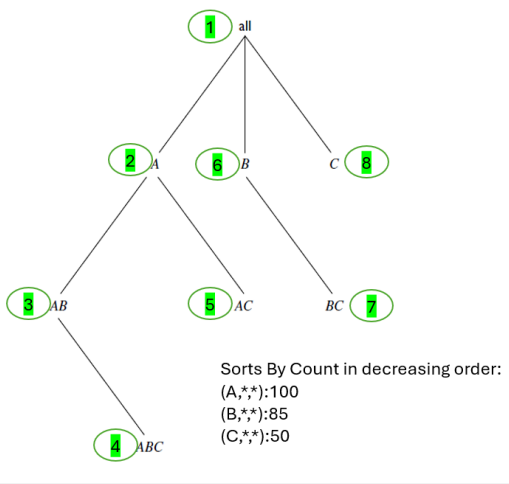


Figure 13: BUC traversal for the computation of the 3-D Cube (A,B,C)

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

Worked Example

Consider the iceberg cube expressed in SQL as follows:

```
1 compute cube iceberg cube as
2 select A, B, C, D, count(*)
3 from R
4 cube by A, B, C, D
5 having count(*) >= 3
```

Sample Iceberg Cube

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

BUC Example

How does BUC constructs the iceberg cube for the dimensions A, B, C, and D, where the minimum support count is 3?

- Suppose that dimension A has four distinct values a_1, a_2, a_3, a_4; B has four distinct values b_1, b_2, b_3, b_4; C has two distinct values c_1, c_2; and D has two distinct values d_1, d_2.
- If we consider each group-by to be a partition, then we must compute every combination of the grouping attributes that satisfy minimum support (ie. that have 3 tuples).
- Figure 14 illustrates how the input is partitioned first according to the different attribute values of dimension A, and then B, C, and D.

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

- To do so, BUC scans the input, aggregating the tuples to obtain a count for **all**, corresponding to the cell (*,*,*,*).
- Dimension A is used to split the input into four partitions, one for each distinct value of A.
- The number of tuples (counts) for each distinct value of A, (a1,*,*,*),(a2,*,*,*),..,(an,*,*,*) is recorded in *dataCount*.

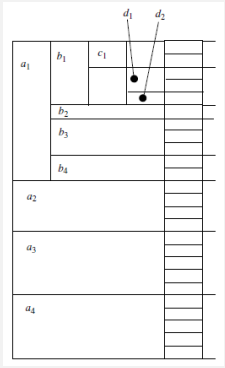


Figure 14: BUC partitioning given a sample 4-D data set

Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

BUC and Apriori (1)

- BUC uses the **Apriori property** for optimisation.
- Starting with A dimension value a_1, the a_1 partition is *aggregated*, creating *one* tuple for the A group-by, corresponding to the cell $(a_1, *, *, *)$.
- If $(a_1, *, *, *)$ satisfies the **minimum support**, a *recursive* call is made on the partition for a_1 and BUC partitions a_1 on dimension B.
- It checks the count of $(a_1, b_1, *, *)$ to see if it satisfies minimum support.
- If it does, it outputs the aggregated tuple to the AB group-by and recurses on $(a_1, b_1, *, *)$ to partition on C, starting with c_1.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

BUC and Apriori (2)

- Assume the cell count for $(a_1, b_1, c_1, *)$ is 2, which *does not satisfy* the minimum support.
- According to Apriori: if a cell does not satisfy minimum support, then neither can any of its descendants.
- Thus, BUC prunes any further exploration of $(a_1, b_1, c_1, *)$ and avoids partitioning the cell on dimension D.
- It backtracks to the a_1, b_1 partition and recurses on $(a_1, b_1, c_2, *)$, and so on.
- By checking the iceberg condition each time before performing a recursive call, BUC saves significant processing time whenever count(cell) $<$ *minsup*.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction

BUC Algorithm

A	Salesperson		Anna	Brian	Claire	Don		
B	Product		Apple	Banana	Cakes	Drinks		
C	City		Toronto	Montreal				
D	Date		Jan	Feb				
A	B	C	D					
*	*	*	*		100			
Anna	*	*	*		25		Go down	
Anna	Apple	*	*		10		Go down	
Anna	Apple	Toronto	*		1		Stop, don't goto Base	
Anna	Apple	Montreal	*		9		Go down	
Anna	Apple	Montreal	Jan		8		At the base, move to next Base element	
			Feb		1		Reject & Go back up, no more cities, go back up again	
Anna	Banana	*	*		2		Go back up	
Anna	Cakes	*	*		12		Fine, so drill down	
Anna	Cakes	Toronto	*		6		Go down	
Anna	Cakes	Toronto	Jan		4		At the base, move to next Base element	
Anna	Cakes	Toronto	Feb		2		Stop, go back up	

Figure 15: Apriori Processing where minsup=3



icient mputation of ta Cubes
derstanding mensionality
tial Materialisation
thods for Data be Computation
l Cube Materialisation
berg Cube terialisation
sed Cube terialisation
tom Up nstruction

BUC and Performance

- The performance of BUC is sensitive to the *order* of the dimensions and to *skew* in the data.
- Ideally, the most discriminating dimensions should be processed first: dimensions should be processed in order of *decreasing* cardinality.
- The higher the cardinality, the smaller the partitions and thus, the more partitions there will be, thereby providing BUC with greater opportunity for pruning.
- Why? With many dimensions (sharing the count), the algorithm will hit "Stop, Go back" quicker and more often.
- Similarly, the more uniform the dimension (having less skew), the better it is for pruning.



Efficient Computation of Data Cubes
Understanding Dimensionality
Partial Materialisation
Methods for Data Cube Computation
Full Cube Materialisation
Iceberg Cube Materialisation
Closed Cube Materialisation
Bottom Up Construction